



Click to add text

# 254383 Algorithm Design and Analysis

อ.พิเศษพงษ์ สุธาพันธ์  
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





# Week 4

## Decrease-and-conquer

อ.พิเศษพงษ์ สุธาพันธ์  
ภาควิชาวิทยาการคอมพิวเตอร์และเทคโนโลยีสารสนเทศ





# Decrease-and-conquer

วัตถุประสงค์

1. รู้วิธีการทำงานแบบ Decrease-and-conquer
2. สามารถนำไปประยุกต์ใช้งานได้



# Decrease-and-conquer

แบ่งได้เป็น 3 กลุ่มหลักคือ

- **Decrease by a constant**
- **Decrease by a constant factor**
- **Variable size decrease**

# Decrease-and-conquer

- การคูณกันของ  $a$  จำนวน  $n$  ตัว
- $a^n$  หรือ  $a_1 * a_2 * \dots * a_n$

เราสามารถทำปัญหาให้ลดลงทีละ 1

คือ  $a^n = a^{n-1} * a$



# Decrease-and-conquer

## Decrease by a constant

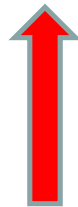
$$f(n) = \begin{cases} f(n-1) \cdot a & \text{if } n > 0, \\ 1 & \text{if } n = 0, \end{cases}$$

Top-down



$f(n - c)$

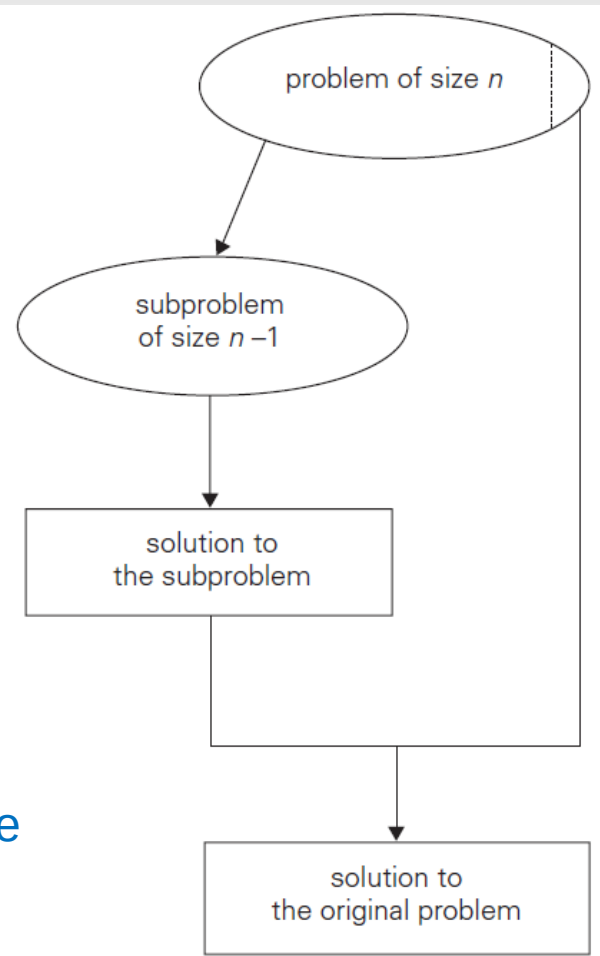
bottom-up



$n * f(1 * c)$

in  $\Theta(n)$

Decrease-(by one)-and-conquer technique





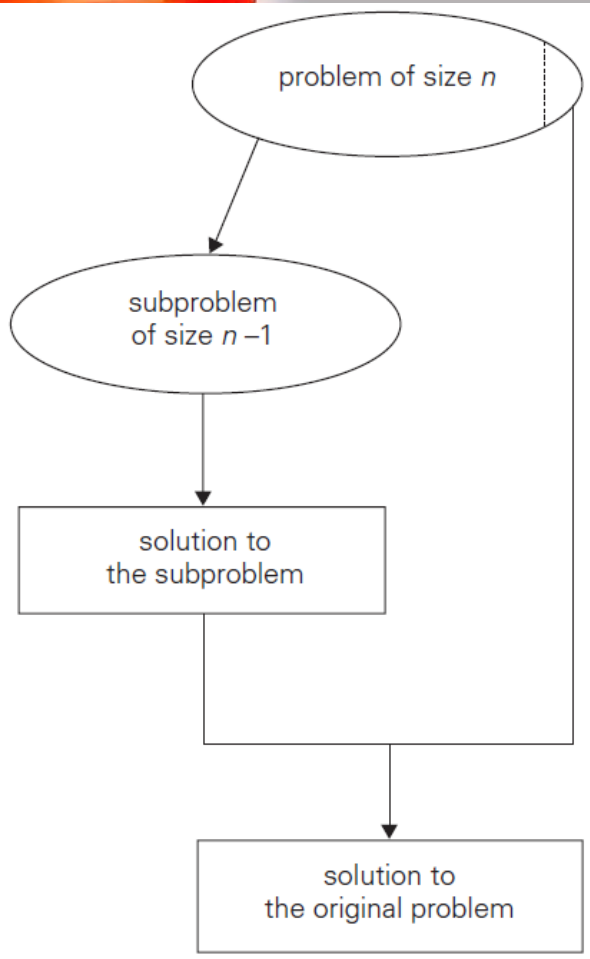
# Decrease by a constant

ตัวอย่าง ปัญหา factorial

$$n! = f(n)$$

$$f(n) = n * f(n-1)$$

$$f(0) = 1$$





# Decrease by a constant

## ตัวอย่าง ปัญหา factorial

$$3! = f(3)$$

$$f(3) = 3 * f(2)$$

$$2 * f(1)$$

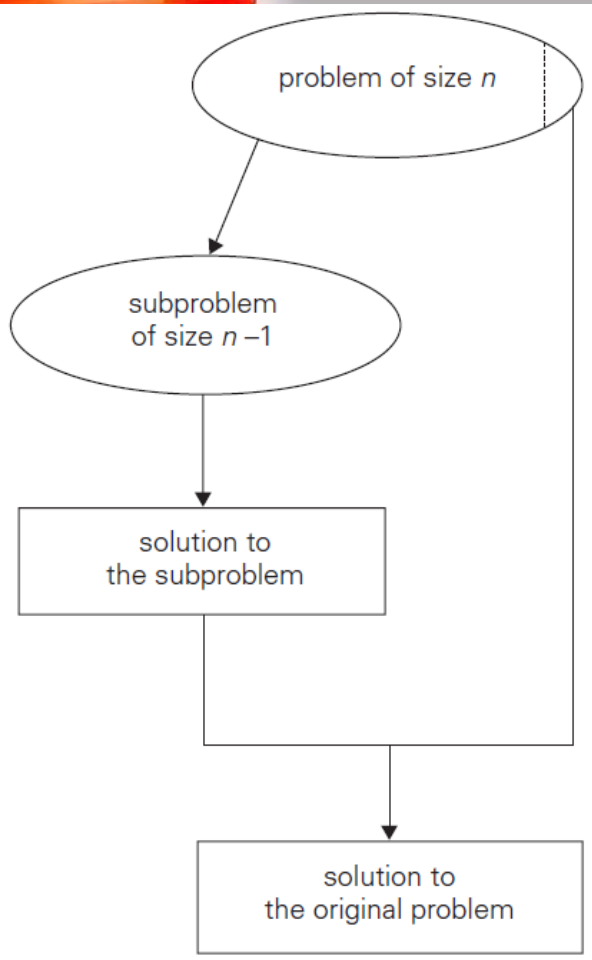
$$1 * f(0)$$

$$1$$

$$6$$

$$2$$

$$1$$





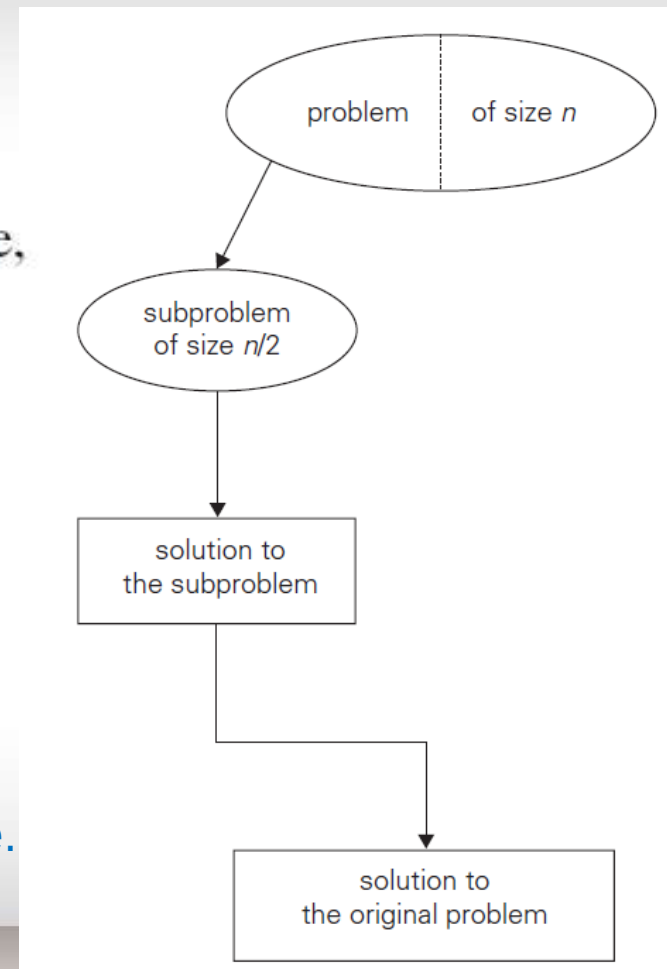
# Decrease-and-conquer

## Decrease by a constant-factor

$$a^n = \begin{cases} (a^{n/2})^2 & \text{if } n \text{ is even and positive,} \\ (a^{(n-1)/2})^2 \cdot a & \text{if } n \text{ is odd,} \\ 1 & \text{if } n = 0. \end{cases}$$

in  $\Theta(\log n)$

Decrease-(by half)-and-conquer technique.





# Decrease by a constant-factor

ตัวอย่าง ปัญหา  $10^5$

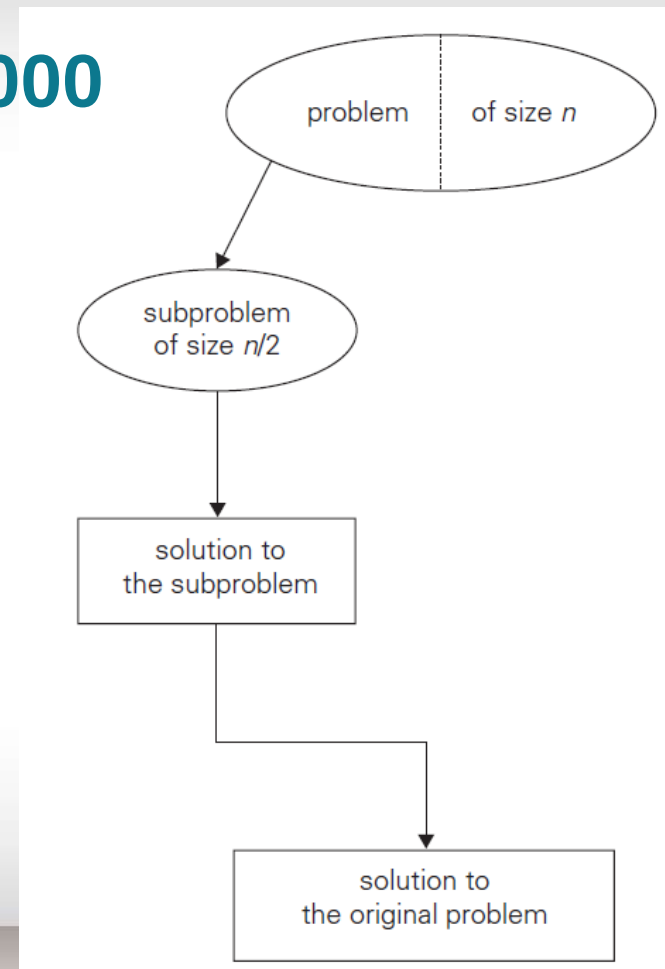
$$10 * 10 * 10 * 10 * 10 = 100000$$

$$10^5 = 10^4 * 10$$

$$= (10^2)^2 * 10$$

$$= (10 * 10)^2 * 10$$

$$= (100 * 100) * 10$$





# Decrease by a constant-factor

ตัวอย่าง ปัญหา  $10^{100}$

$$10^{100} = (10^{50})^2$$

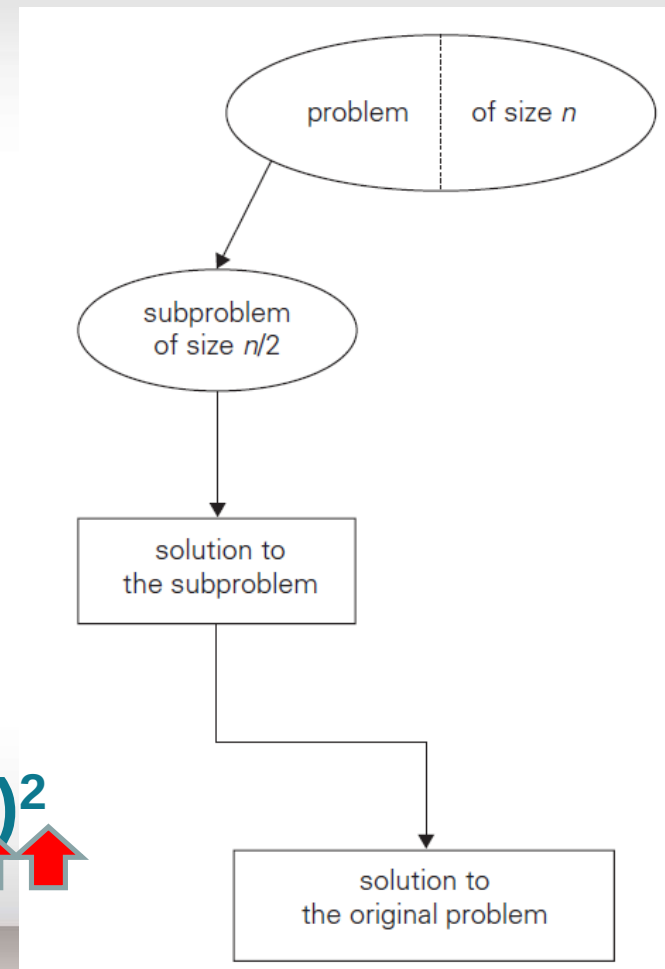
$$= ((10^{25})^2)^2$$

$$= (((10^{12})^2 * 10)^2)^2$$

$$= (((((10^6)^2)^2 * 10)^2)^2)^2$$

$$= ((((((10^3)^2)^2)^2 * 10)^2)^2)^2)^2$$

$$= ((((((10^2 * 10)^2)^2)^2 * 10)^2)^2)^2)^2$$





# Decrease-and-conquer

## Variable-size-decrease

๓.๒.

$$\gcd(m, n) = \gcd(n, m \bmod n).$$



# Variable-size-decrease

ต.ย.

$$\text{gcd}(270, 24)$$

$$= (24, 6)$$

$$= (6, 0)$$

$$\text{gcd} = 6$$



# Decrease-and-conquer

## 4.1 Insertion Sort



# Insertion Sort

**ALGORITHM** *InsertionSort*( $A[0..n - 1]$ )

//Sorts a given array by insertion sort

//Input: An array  $A[0..n - 1]$  of  $n$  orderable elements

//Output: Array  $A[0..n - 1]$  sorted in nondecreasing order

**for**  $i \leftarrow 1$  **to**  $n - 1$  **do**

$v \leftarrow A[i]$

$j \leftarrow i - 1$

**while**  $j \geq 0$  **and**  $A[j] > v$  **do**

$A[j + 1] \leftarrow A[j]$

$j \leftarrow j - 1$

$A[j + 1] \leftarrow v$



# Insertion Sort

32

7 , 32 , 6 , 38 , 33 ,



6

7 , 32 , 6 , 38 , 33 ,



6

7 , 32 , 32 , 38 , 33 ,



# Insertion Sort

6

7 , 32 , 32 , 38 , 33 ,

6



7 , 7 , 32 , 38 , 33 ,

6



6 , 7 , 32 , 38 , 33 ,

38



6 , 7 , 32 , 38 , 33 ,



# Insertion Sort

33

6 , 7 , 32 , 38 , 33 ,



33

6 , 7 , 32 , 38 , 38 ,



33

6 , 7 , 32 , 33 , 38 ,





# Insertion Sort

$A[0] \leq \dots \leq A[j] < A[j+1] \leq \dots \leq A[i-1] \mid A[i] \dots A[n-1]$

smaller than or equal to  $A[i]$       greater than  $A[i]$



# Insertion Sort

$$C_{worst}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \in \Theta(n^2).$$

$$C_{best}(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n).$$

ข้อมูลเรียงจากน้อยไปมาก

$$C_{avg}(n) \approx \frac{n^2}{4} \in \Theta(n^2).$$



# Decrease-and-conquer

## 4.2 Topological Sorting



# Decrease-and-conquer

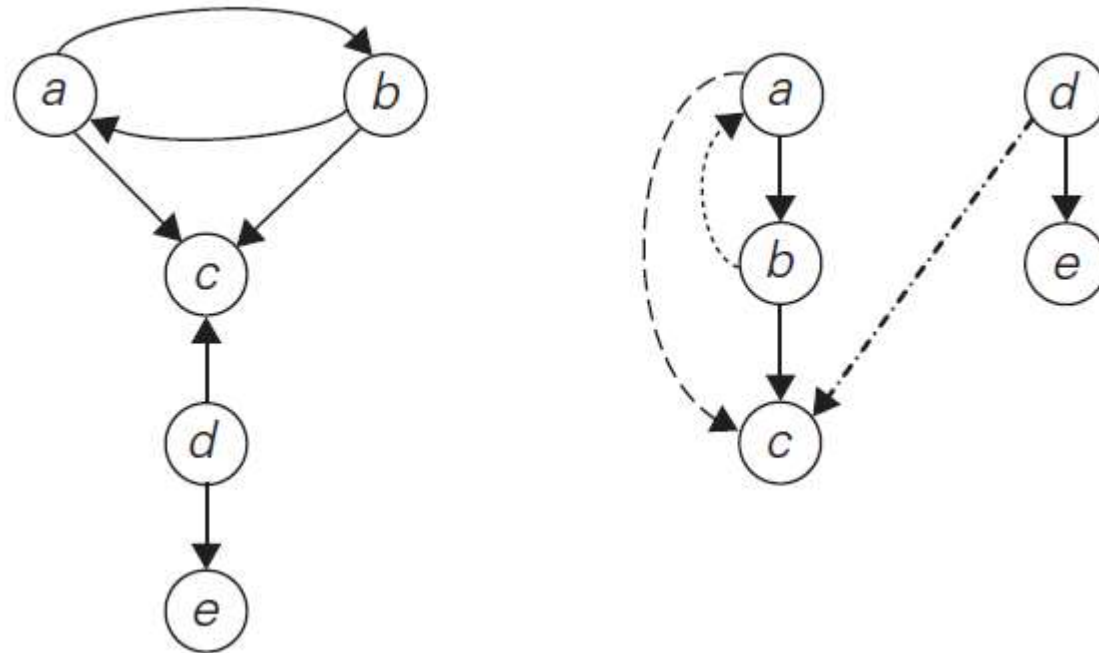
## 4.2 Topological Sorting

วิธีที่นำมาใช้

1. Depth First Search (DFS)
2. Source Removal



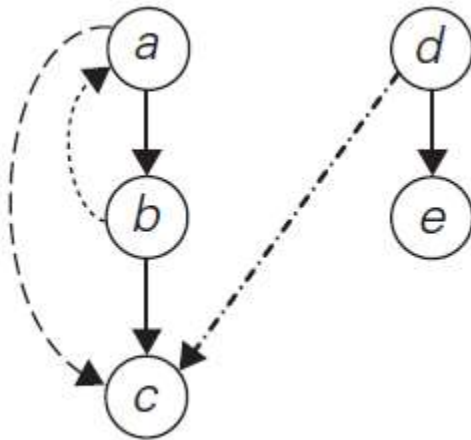
# Topological Sorting



(a) Digraph. (b) DFS forest of the digraph for the DFS traversal started at  $a$ .



# Topological Sorting



**four types of edges possible in a DFS forest of a directed graph:**

***tree edges (ab, bc, de)***

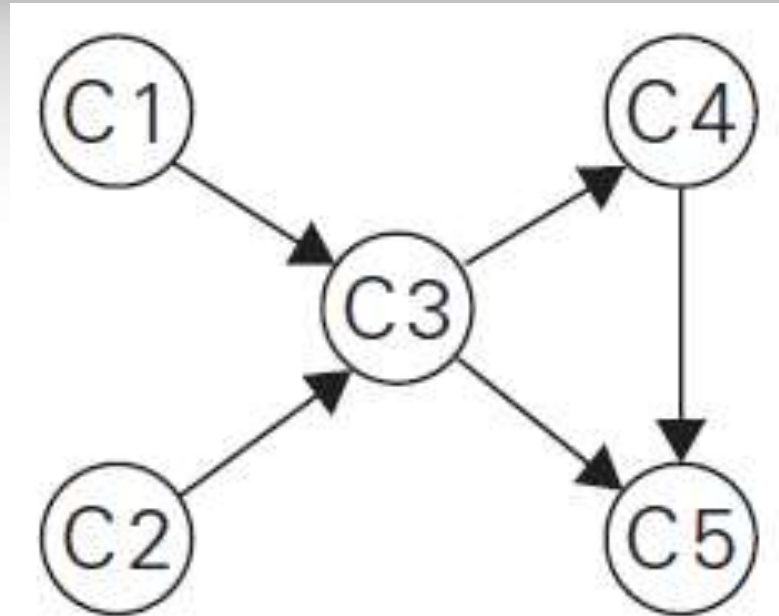
***Back edges (ba)***

***forward edges (ac)***

***cross edges (dc)***



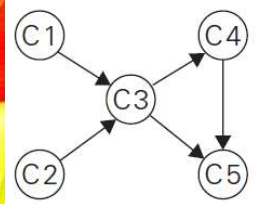
# Topological Sorting



Digraph representing the prerequisite structure of five courses



# Topological Sorting



C5<sub>1</sub>

C4<sub>2</sub>

C3<sub>3</sub>

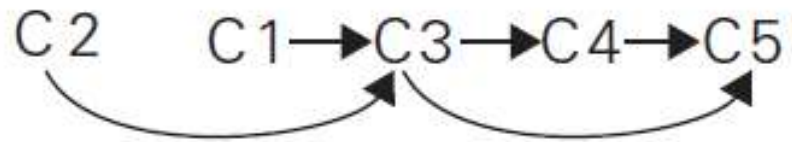
C1<sub>4</sub> C2<sub>5</sub>

(b)

The popping-off order:

C5, C4, C3, C1, C2

The topologically sorted list:



(c)

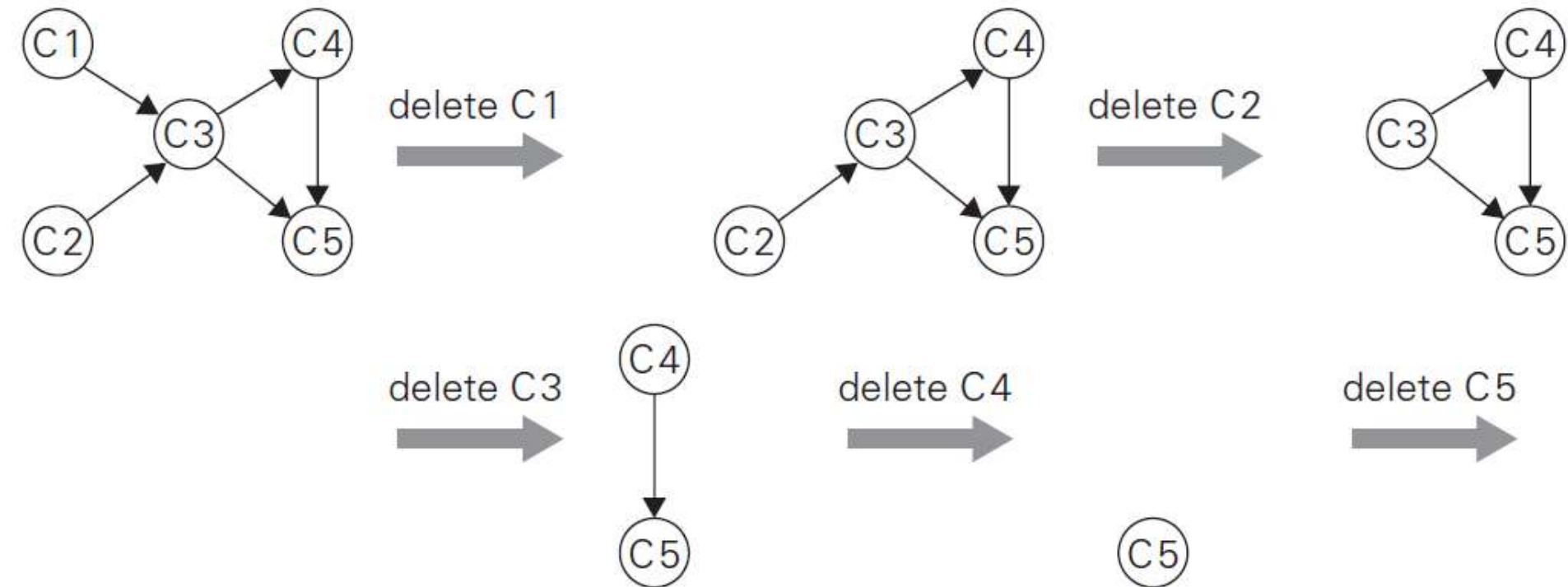
(a) Digraph for which the topological sorting problem needs to be solved.

(b) **DFS** traversal stack with the subscript numbers indicating the poppingoff order.

(c) Solution to the problem.



# Topological Sorting



The solution obtained is C1, C2, C3, C4, C5

(a) Digraph for which the topological sorting problem Illustration of the **source-removal** algorithm for the topological sorting problem. On each iteration, a vertex with no incoming edges is deleted from the digraph.



# **Decrease-and-conquer**

## **4.3 Algorithms for Generating Combinatorial Objects**



# Generating Permutations

## Minimal-change requirement

|                                |     |     |     |
|--------------------------------|-----|-----|-----|
| start                          | 1   |     |     |
| insert 2 into 1 right to left  | 12  | 21  |     |
| insert 3 into 12 right to left | 123 | 132 | 312 |
| insert 3 into 21 left to right | 321 | 231 | 213 |

Generating permutations bottom up



# Generating Permutations

## Johnson-Trotter algorithm

**ALGORITHM** *JohnsonTrotter*( $n$ )

//Implements Johnson-Trotter algorithm for generating permutations

//Input: A positive integer  $n$

//Output: A list of all permutations of  $\{1, \dots, n\}$

initialize the first permutation with  $\overleftarrow{1} \overleftarrow{2} \dots \overleftarrow{n}$

**while** the last permutation has a mobile element **do**

    find its largest mobile element  $k$

    swap  $k$  with the adjacent element  $k$ 's arrow points to

    reverse the direction of all the elements that are larger than  $k$

    add the new permutation to the list



# Generating Permutations

## Johnson-Trotter algorithm

หาค่าที่เคลื่อนที่ได้ที่มีค่ามากที่สุด( $k$ )โดยดูทิศทางจากลูกศร สลับ  $k$  กับตัวที่ติดกันตามทิศของ  $K$  แล้วเปลี่ยนทิศทางทุกตัวในกรณีที่ตัวนั้นมีค่ามากกว่า  $k$

<http://10.31.26.28/~phisetphong/algorithm/johnsontrotter.html>



# Generating Permutations

## Johnson-Trotter algorithm

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 1 | 2 | 4 | 3 |
| 1 | 4 | 2 | 3 |
| 4 | 1 | 2 | 3 |
| 4 | 1 | 3 | 2 |
| 1 | 4 | 3 | 2 |
| 1 | 3 | 4 | 2 |
| 1 | 3 | 2 | 4 |

|   |   |   |   |
|---|---|---|---|
| 3 | 1 | 2 | 4 |
| 3 | 1 | 4 | 2 |
| 3 | 4 | 1 | 2 |
| 4 | 3 | 1 | 2 |
| 4 | 3 | 2 | 1 |
| 3 | 4 | 2 | 1 |
| 3 | 2 | 4 | 1 |
| 3 | 2 | 1 | 4 |

|   |   |   |   |
|---|---|---|---|
| 2 | 3 | 1 | 4 |
| 2 | 3 | 4 | 1 |
| 2 | 4 | 3 | 1 |
| 4 | 2 | 3 | 1 |
| 4 | 2 | 1 | 3 |
| 2 | 4 | 1 | 3 |
| 2 | 1 | 4 | 3 |
| 2 | 1 | 3 | 4 |



# Generating Permutations

## Johnson-Trotter algorithm

- $\langle 1 \ 2 \ 3 \ 4$
- $\langle 1 \ 2 \ 4 \ 3$
- $\langle 1 \ 4 \ 2 \ 3$
- $\langle 4 \ 1 \ 2 \ 3$
- $4 \rangle \ 1 \ 3 \ 2$
- $\langle 1 \ 4 \rangle \ 3 \ 2$
- **$\langle 1 \ 3 \ 4 \rangle \ 2$**
- $\langle 1 \ 3 \ 2 \ 4 \rangle$

- $\langle 3 \ 1$
- $\langle 2 \ 4$
- $\langle 3 \ 1$
- $\langle 4 \ 2$
- $\langle 3 \ 4$
- $\langle 1 \ 2$
- $\langle 4 \ 3$
- $\langle 1 \ 2$
- $4 \rangle \ 3 \rangle$

- $\langle 2 \ 3 \rangle \ 1$
- $\langle 4$
- $\langle 2 \ 3 \rangle \ 4$
- $\langle 1$
- $\langle 2 \ 4 \ 3 \rangle$
- $\langle 1$
- $\langle 4 \ 2 \ 3 \rangle$
- $\langle 1$
- $4 \rangle \ 2 \ 1$



# Generating Permutations

## lexicographic order

**ALGORITHM** *LexicographicPermute*( $n$ )

//Generates permutations in lexicographic order

//Input: A positive integer  $n$

//Output: A list of all permutations of  $\{1, \dots, n\}$  in lexicographic order

initialize the first permutation with  $12 \dots n$

**while** last permutation has two consecutive elements in increasing order **do**

    let  $i$  be its largest index such that  $a_i < a_{i+1}$  //  $a_{i+1} > a_{i+2} > \dots > a_n$

    find the largest index  $j$  such that  $a_i < a_j$  //  $j \geq i + 1$  since  $a_i < a_{i+1}$

    swap  $a_i$  with  $a_j$  //  $a_{i+1}a_{i+2} \dots a_n$  will remain in decreasing order

    reverse the order of the elements from  $a_{i+1}$  to  $a_n$  inclusive

    add the new permutation to the list



# Generating Permutations

## lexicographic order

หาตัวอักษรที่ติดกันที่เรียงจากน้อยไปมาก  
โดยให้ ตำแหน่ง  $i$  มีค่ามากที่สุดและ  $a_i <$   
 $a_{i+1}$

หาตัวตำแหน่ง  $j$  ที่มีค่ามากที่สุด ที่  $a_j$  มีค่า  
มากกว่า  $a_i$

สลับตำแหน่ง  $a_i$  และ  $a_j$  แล้ว จะเกิดกลุ่ม  
ข้อมูลที่เรียงจากมากไปน้อย ให้ทำการ  
reverse ข้อมูล



# Generating Permutations

lexicographic order

ABCD

ABDC

.....

DCAB

DCBA



$a_i$  ABCD จากเงื่อนไข  $i$  มากที่สุด ที่  $a_i < a_{i+1}$   
 $a_j$  ABCD จากเงื่อนไข  $j$  มากที่สุด ที่  $a_i < a_j$   
 หามสลับ  $a_i$  และ  $a_j$  ทุกค่าที่เรียงจากมากไปน้อยทั้งหมด Reverse  
 $\rightarrow$  ABDC

$A \overset{i}{B} D \overset{j}{C} \rightarrow A C \underline{D} B \rightarrow A C B D *$



# Generating Permutations

## Binary Reflected Gray Code

**ALGORITHM**  $BRGC(n)$

//Generates recursively the binary reflected Gray code of order  $n$

//Input: A positive integer  $n$

//Output: A list of all bit strings of length  $n$  composing the Gray code

**if**  $n = 1$  make list  $L$  containing bit strings 0 and 1 in this order

**else** generate list  $L1$  of bit strings of size  $n - 1$  by calling  $BRGC(n - 1)$

    copy list  $L1$  to list  $L2$  in reversed order

    add 0 in front of each bit string in list  $L1$

    add 1 in front of each bit string in list  $L2$

    append  $L2$  to  $L1$  to get list  $L$

**return**  $L$



# Generating Permutations

## Binary Reflected Gray Code

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 1 |
| 0 | 1 | 1 |
| 0 | 1 | 0 |
| 1 | 1 | 0 |
| 1 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 0 | 0 |

ข้อมูลในลำดับจะมี  
การเปลี่ยนแปลง  
เพียงตัวเดียวเท่านั้น  
ในลำดับถัดไป

ประโยชน์ใช้ในการควบคุมสวิตซ์การทำงานโดยการเปลี่ยนแปลงเพียงอันเดียว



# Generating Permutations

## Binary Reflected Gray Code

| <u>Gray codes</u> |   |   | Binary numbers |   |       |
|-------------------|---|---|----------------|---|-------|
| 0                 | 0 | 0 | —              | 0 | 0 0 0 |
| 0                 | 0 | 1 | —              | 1 | 0 0 1 |
| 0                 | 1 | 1 | +              | 2 | 0 1 0 |
| 0                 | 1 | 0 | +              | 3 | 0 1 1 |
| 1                 | 1 | 0 | +              | 4 | 1 0 0 |
| 1                 | 1 | 1 | +              | 5 | 1 0 1 |
| 1                 | 0 | 1 | +              | 6 | 1 1 0 |
| 1                 | 0 | 0 | +              | 7 | 1 1 1 |



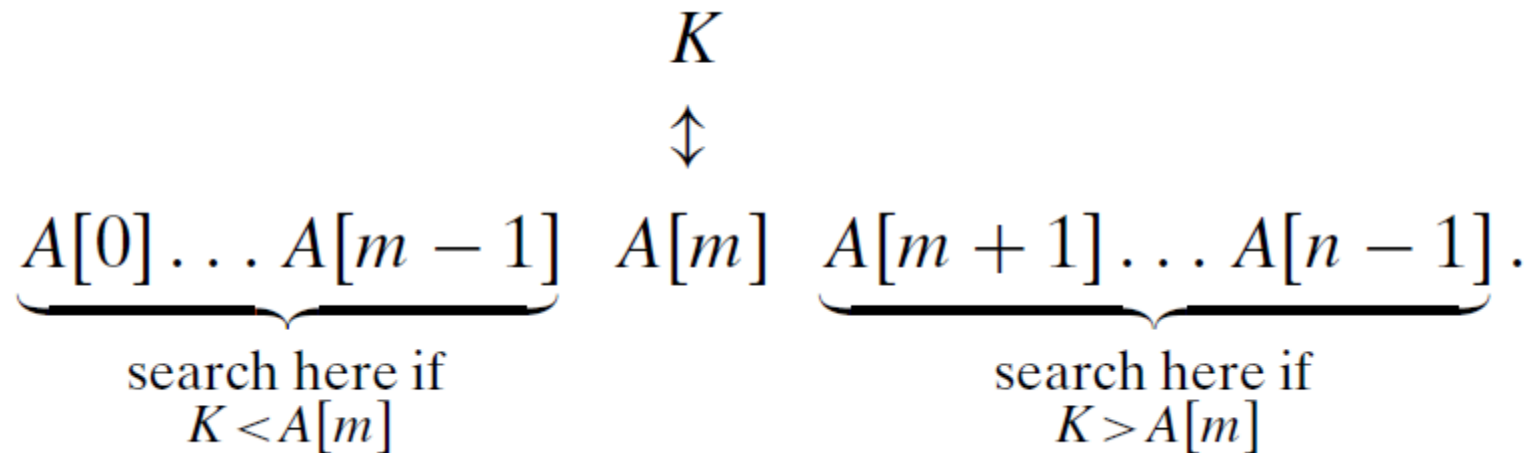
# Decrease-and-conquer

## 4.4 Decrease-by-a-Constant-Factor Algorithms



# Decrease-by-a-Constant-Factor

## Binary Search



$$K = 70$$

|   |    |    |    |    |    |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|----|----|----|----|----|
| 3 | 14 | 27 | 31 | 39 | 42 | 55 | 70 | 74 | 81 | 85 | 93 | 98 |
|---|----|----|----|----|----|----|----|----|----|----|----|----|



# Decrease-by-a-Constant-Factor

## Binary Search

| index       | 0   | 1  | 2  | 3  | 4  | 5  | 6   | 7      | 8   | 9   | 10 | 11 | 12  |
|-------------|-----|----|----|----|----|----|-----|--------|-----|-----|----|----|-----|
| value       | 3   | 14 | 27 | 31 | 39 | 42 | 55  | 70     | 74  | 81  | 85 | 93 | 98  |
| iteration 1 | $l$ |    |    |    |    |    | $m$ |        |     |     |    |    | $r$ |
| iteration 2 |     |    |    |    |    |    |     | $l$    |     | $m$ |    |    | $r$ |
| iteration 3 |     |    |    |    |    |    |     | $l, m$ | $r$ |     |    |    |     |



# Decrease-by-a-Constant-Factor

## Binary Search

**ALGORITHM** *BinarySearch*( $A[0..n - 1]$ ,  $K$ )

//Implements nonrecursive binary search

//Input: An array  $A[0..n - 1]$  sorted in ascending order and  
// a search key  $K$

//Output: An index of the array's element that is equal to  $K$   
// or  $-1$  if there is no such element

$l \leftarrow 0$ ;  $r \leftarrow n - 1$

**while**  $l \leq r$  **do**

$m \leftarrow \lfloor (l + r)/2 \rfloor$

**if**  $K = A[m]$  **return**  $m$

**else if**  $K < A[m]$   $r \leftarrow m - 1$

**else**  $l \leftarrow m + 1$

**return**  $-1$



# Decrease-by-a-Constant-Factor

## Binary Search

$$C_{worst}(n) = C_{worst}(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1, \quad C_{worst}(1) = 1.$$

$$C_{worst}(2^k) = k + 1 = \log_2 n + 1.$$

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2(n + 1) \rceil.$$

$$\Theta(\log n)$$



# Decrease-by-a-Constant-Factor

## Binary Search

$$C_{avg}(n) \approx \log_2 n.$$



# Decrease-by-a-Constant-Factor

## Fake-Coin Problem

มีเหรียญอยู่กองหนึ่ง ซึ่งมีเหรียญปลอมอยู่ในนั้น 1 เหรียญ โดยเหรียญปลอมน้ำหนักน้อยกว่าเหรียญจริง โดยให้ใช้ตาชั่งแบบสมดุลได้ จะมีวิธีการหาอย่างไร

$$W(n) = W(\lfloor n/2 \rfloor) + 1 \quad \text{for } n > 1, \quad W(1) = 0.$$



# Decrease-by-a-Constant-Factor

## Russian Peasant Multiplication

$$n \cdot m = \frac{n}{2} \cdot 2m.$$

$n$  is even

$$n \cdot m = \frac{n-1}{2} \cdot 2m + m.$$

$n$  is odd



# Decrease-by-a-Constant-Factor

**Russian Peasant Method  
of Multiplication**

with Mr. Marcos



# Decrease-by-a-Constant-Factor

## Russian Peasant Multiplication

| $n$ | $m$  |                        | $n$ | $m$  |             |
|-----|------|------------------------|-----|------|-------------|
| 50  | 65   |                        | 50  | 65   |             |
| 25  | 130  |                        | 25  | 130  | 130         |
| 12  | 260  | (+130)                 | 12  | 260  |             |
| 6   | 520  |                        | 6   | 520  |             |
| 3   | 1040 |                        | 3   | 1040 | 1040        |
| 1   | 2080 | (+1040)                | 1   | 2080 | 2080        |
|     | 2080 | $+(130 + 1040) = 3250$ |     |      | <u>3250</u> |

Computing  $50 \cdot 65$  by the Russian peasant method.



# Decrease-by-a-Constant-Factor

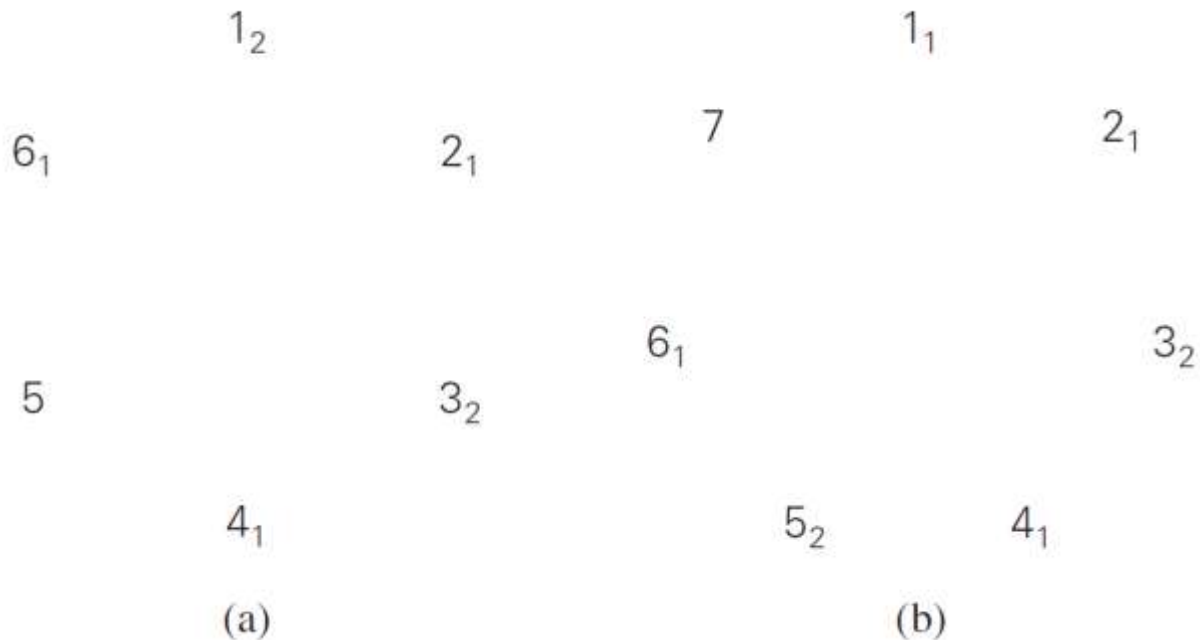
## Josephus Problem

ให้  $n$  คน หมายเลข 1 ถึง  $n$  ยืนอยู่ในวงกลม เริ่มต้นนับจากคนที่ 1 และจะฆ่าทุกคนที่อยู่ในตำแหน่งที่สองจนเหลือเพียงคนเดียว



# Decrease-by-a-Constant-Factor

## Josephus Problem



Instances of the Josephus problem for (a)  $n = 6$  and (b)  $n = 7$ . Subscript numbers indicate the pass on which the person in that position is eliminated. The solutions are  $J(6) = 5$  and  $J(7) = 7$ , respectively.



# Decrease-by-a-Constant-Factor

## Josephus Problem

กรณี  $n$  เป็น

$$J(2k) = 2J(k) - 1$$

$$n = 2k$$

กรณี  $n$  เป็น

$$J(2k + 1) = 2J(k) + 1$$

$$n = 2k + 1$$

เลขคี่

กำหนด

$$J(1) = 1$$



# Decrease-by-a-Constant-Factor

## Josephus Problem

$$n = 6$$

$$J(2k) = 2J(k) - 1$$

$$J(2*3) = 2J(3) - 1$$

$$k = 3$$

$$J(2*1 + 1) = 2J(1) + 1$$

$$k = 1$$

$$J(1) = 1$$

ใช้ backward substitutions

1



2\*1+1



2(2\*1+1)-1 = 5



# Decrease-by-a-Constant-Factor

## Josephus Problem

แบบลัด

*แทน  $n$  ด้วยเลขฐาน 2*

*ทำ 1-bit cyclic shift left*

For example,

$$J(6) = J(110_2) = 101_2 = 5$$

$$J(7) = J(111_2) = 111_2 = 7.$$



# **Decrease-and-conquer**

## **4.5 Variable-Size-Decrease Algorithms**



# Variable-Size-Decrease

## Computing a Median and the Selection Problem

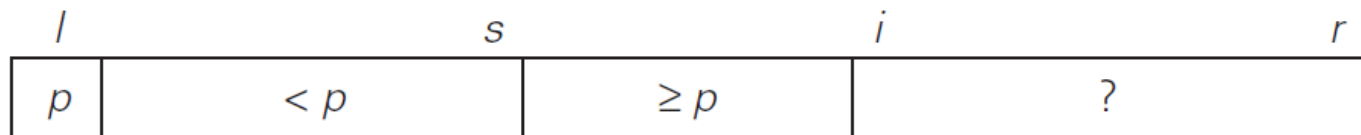


*pivot*  $p$

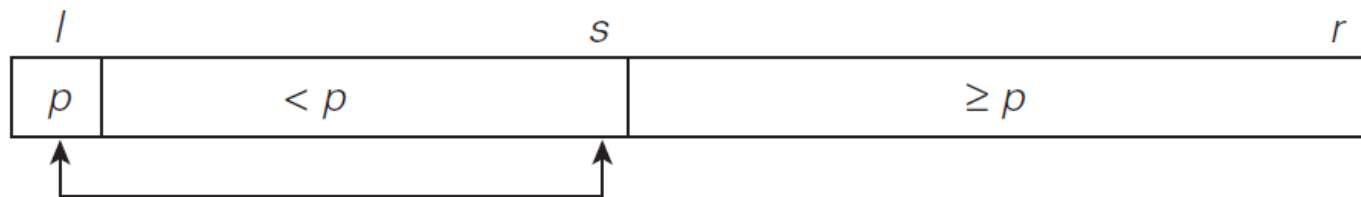


# Variable-Size-Decrease

## *Lomuto partitioning*



(a)



(b)



(c)



# Variable-Size-Decrease

**ALGORITHM** *LomutoPartition*( $A[l..r]$ )

//Partitions subarray by Lomuto's algorithm using first element as pivot

//Input: A subarray  $A[l..r]$  of array  $A[0..n - 1]$ , defined by its left and right

// indices  $l$  and  $r$  ( $l \leq r$ )

//Output: Partition of  $A[l..r]$  and the new position of the pivot

$p \leftarrow A[l]$

$s \leftarrow l$

**for**  $i \leftarrow l + 1$  **to**  $r$  **do**

**if**  $A[i] < p$

$s \leftarrow s + 1$ ; swap( $A[s]$ ,  $A[i]$ )

swap( $A[l]$ ,  $A[s]$ )

**return**  $s$



# Variable-Size-Decrease

**ALGORITHM** *Quickselect*( $A[l..r]$ ,  $k$ )

//Solves the selection problem by recursive partition-based algorithm  
//Input: Subarray  $A[l..r]$  of array  $A[0..n - 1]$  of orderable elements and  
// integer  $k$  ( $1 \leq k \leq r - l + 1$ )  
//Output: The value of the  $k$ th smallest element in  $A[l..r]$   
 $s \leftarrow \text{LomutoPartition}(A[l..r])$  //or another partition algorithm  
**if**  $s = k - 1$  **return**  $A[s]$   
**else if**  $s > l + k - 1$  *Quickselect*( $A[l..s - 1]$ ,  $k$ )  
**else** *Quickselect*( $A[s + 1..r]$ ,  $k - 1 - s$ )



# Variable-Size-Decrease

ต.ย. หา median ของตัวเลข 9

ตัว

4, 1, 10, 8, 7, 12, 9, 2, 15

จะมี median อยู่ที่ตำแหน่งที่  $k = \lceil 9/2 \rceil = 5$  ในลำดับที่เรียง

แล้ว

1, 2, 4, 7, 8, 9, 10, 12, 15



# Variable-Size-Decrease

| 0        | 1        | 2        | 3 | 4 | 5  | 6 | 7        | 8        |
|----------|----------|----------|---|---|----|---|----------|----------|
| <hr/>    |          |          |   |   |    |   |          |          |
| <i>s</i> | <i>i</i> |          |   |   |    |   |          |          |
| <b>4</b> | 1        | 10       | 8 | 7 | 12 | 9 | 2        | 15       |
|          | <i>s</i> | <i>i</i> |   |   |    |   |          |          |
| <b>4</b> | 1        | 10       | 8 | 7 | 12 | 9 | 2        | 15       |
|          | <i>s</i> |          |   |   |    |   | <i>i</i> |          |
| <b>4</b> | 1        | 10       | 8 | 7 | 12 | 9 | 2        | 15       |
|          |          | <i>s</i> |   |   |    |   | <i>i</i> |          |
| <b>4</b> | 1        | 2        | 8 | 7 | 12 | 9 | 10       | 15       |
|          |          | <i>s</i> |   |   |    |   |          | <i>i</i> |
| <b>4</b> | 1        | 2        | 8 | 7 | 12 | 9 | 10       | 15       |
| 2        | 1        | <b>4</b> | 8 | 7 | 12 | 9 | 10       | 15       |

Since  $s = 2$  is smaller than  $k - 1 = 4$ , we proceed with the right part of the array



# Variable-Size-Decrease

| 0 | 1 | 2 | 3        | 4        | 5        | 6 | 7  | 8        |
|---|---|---|----------|----------|----------|---|----|----------|
|   |   |   | <i>s</i> | <i>i</i> |          |   |    |          |
|   |   |   | <b>8</b> | 7        | 12       | 9 | 10 | 15       |
|   |   |   |          | <i>s</i> | <i>i</i> |   |    |          |
|   |   |   | <b>8</b> | 7        | 12       | 9 | 10 | 15       |
|   |   |   |          | <i>s</i> |          |   |    | <i>i</i> |
|   |   |   | <b>8</b> | 7        | 12       | 9 | 10 | 15       |
|   |   |   | 7        | <b>8</b> | 12       | 9 | 10 | 15       |

Now  $s = k - 1 = 4$ , and hence we can stop: the found median is 8, which is greater than 2, 1, 4, and 7 but smaller than 12, 9, 10, and 15.



# Variable-Size-Decrease

**ประสิทธิภาพ ของ  
Quickselect**

$$C_{best}(n) = n - 1 \in \Theta(n)$$

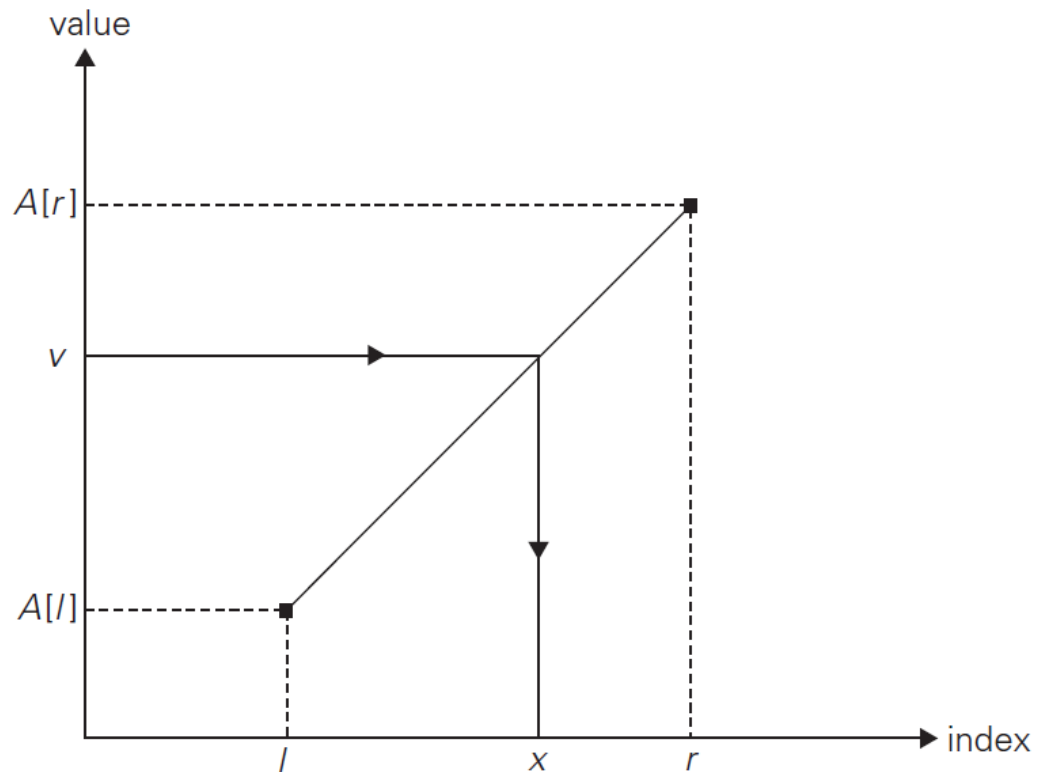
$$C_{worst}(n) = (n - 1) + (n - 2) + \dots + 1 = (n - 1)n/2 \in \Theta(n^2),$$



# Variable-Size-Decrease

## *Interpolation Search*

### Algorithm for searching in a sorted array



Index computation in interpolation search.



# Variable-Size-Decrease

## *Interpolation Search*

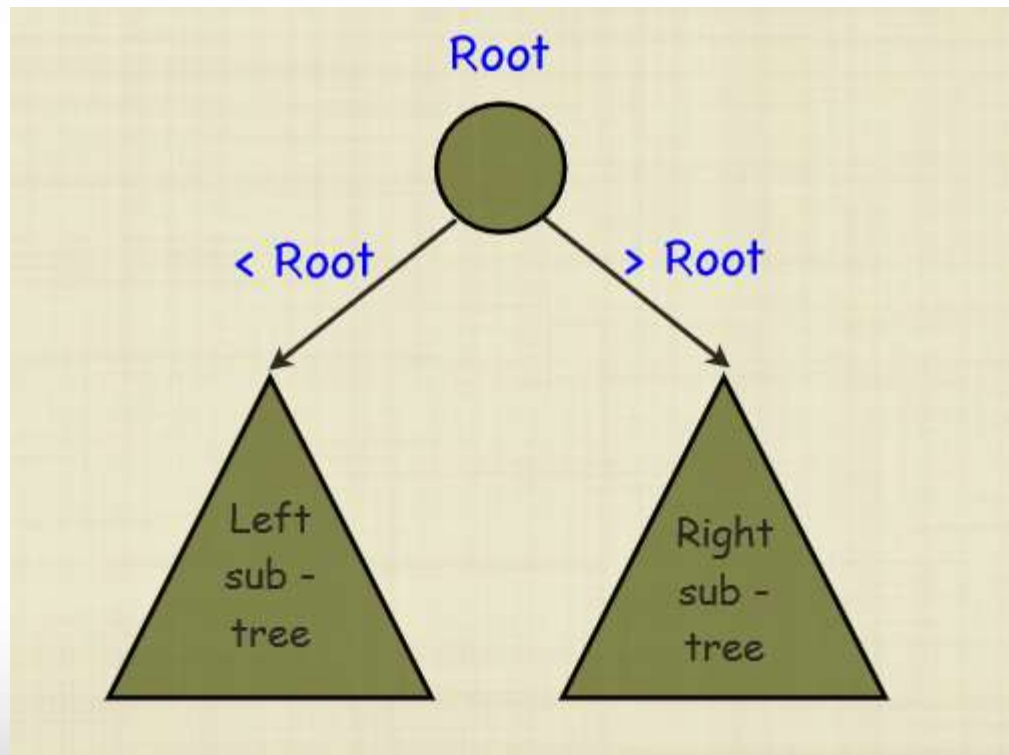
เป็นการค้นหาในข้อมูลที่มีการจัดทำ index แล้ว  
เช่น การค้นหารายชื่อในสมุดหน้าเหลือง



# Variable-Size-Decrease

## *Searching and Insertion in a Binary Search Tree*

### Binary Search Tree





# Variable-Size-Decrease

## *Searching and Insertion in a Binary Search Tree*

### Binary Search Tree

worst-case efficiency of the search operation fall into  $\Theta(n)$

average-case efficiency turns out to be in  $\Theta(\log n)$

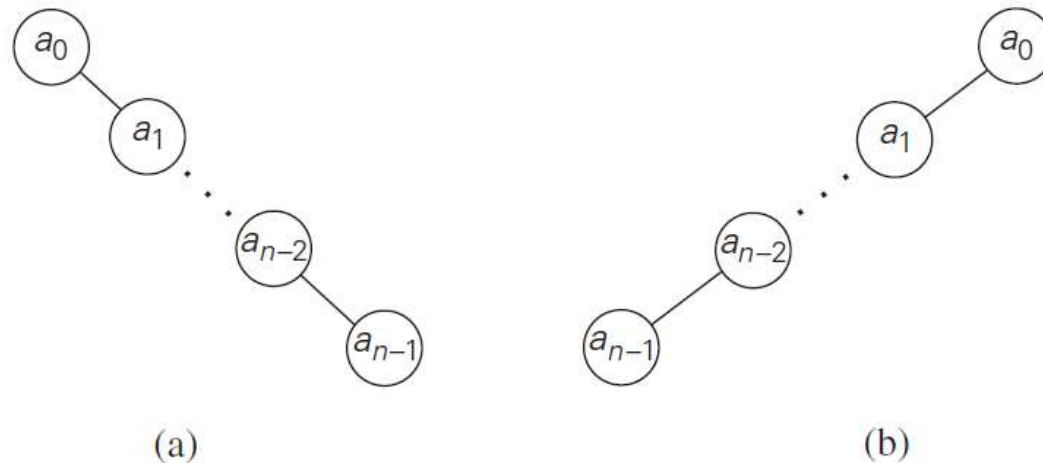


# Variable-Size-Decrease

## *Searching and Insertion in a Binary Search Tree*

### Binary Search Tree

ต.ย. ข้อมูลที่จัดเรียงแล้วมาสร้างเป็น binary search tree



Binary search trees for (a) an increasing sequence of keys and (b) a decreasing sequence of keys



# Variable-Size-Decrease

## The Game of Nim

นิม (อังกฤษ: Nim) คือ เกมวางแผนทางคณิตศาสตร์เกมหนึ่ง ซึ่งมีผู้เล่น 2 คน และมีกติกาในการเล่นดังนี้

1. มีกองหินอยู่หลายกอง แต่ละกองมีก้อนหินอยู่
2. ผู้เล่นจะผลัดกันหยิบก้อนหิน
3. ในการหยิบแต่ละครั้ง ผู้เล่นจะต้องหยิบก้อนหินออกจากกองหินกองใดกองหนึ่ง และต้องหยิบอย่างน้อย 1 ก้อน
4. ผู้เล่นที่หยิบก้อนหินก้อนสุดท้าย จะเป็นผู้ชนะ (ผู้ที่หยิบต่อไม่ได้เป็นผู้แพ้)

# การบ้าน

1. เขียนโปรแกรม เกม Nim ด้วยภาษา python โดยให้รับ input เป็นการหยิบหินของผู้เล่น กองใด ก็ก้อน เช่น 1 3 เป็นต้น โดยกองหินและจำนวนหินในกองให้ใช้วิธีการสุ่ม



# อ้างอิงและต้นฉบับ

[1] Introduction to the design & analysis of algorithm, 3<sup>rd</sup>,  
Anany Levitin